

CIS 613: Software Testing

1

How do you test?

2

What is a *successful* test?

3

Software Testing

Testing is the process of executing a program with the intent of finding errors.

What is a *successful* test?

Commonly, we *think* of a successful test as a *passing* test. However...

A *successful* test one that establishes the presence of *one or more* errors in the software.

Glenford Myers, *The Art of Software Testing*

5

Verification vs Validation

- **Verification:** A set of activities that ensure that the software conforms to its specifications:

*"Are we building the thing **right**?"*

- **Validation:** A set of activities that ensure that the software built meets the customer's *needs and expectations*.

*"Are we building the **right** thing?"*

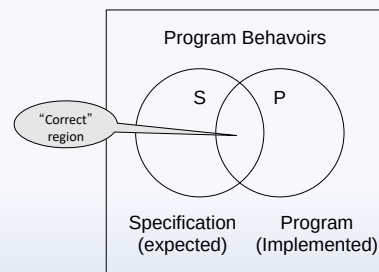
Verification and Validation (V&V) should establish confidence that the software is "fit for purpose"

- This does *not* mean the software is *completely* free of defects.
- It *does mean* the software must be good enough for its intended use, where that use determines the degree of confidence that is needed.

This class focuses on **verification** based on correct specifications.

6

The Reality of Programs



7

On Correctness

- Testing can *only* demonstrate the presence of errors in a program, *not their absence*.
- Simply because a test suite (i.e., a collection of tests) for a program doesn't detect any errors (i.e., failing tests), doesn't mean the program is error free.
- It could easily be that the test suite never exercised, or tested, the program in such a way to indicate that there is an error.
- Realistically, large programs are never* bug free.

*Or, at least, very rarely

8

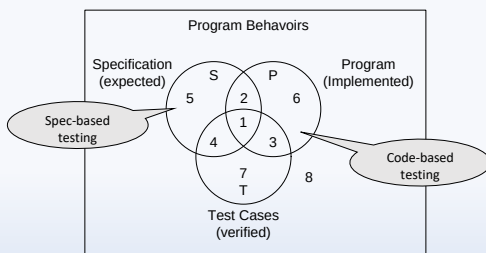
A Better Viewpoint

- Correctness is impossible to demonstrate*
- Correctness is a term from "classical" computer science
 - "proofs" derived from code,
 - Not derived from specification,
 - Therefore, can only prove that the code does what it does!
- **Better Viewpoint:** A relative term—program P is correct with respect to specification S.
- **Bottom Line:** Do the specification and the program meet the customer/user's expectations?

*Or, at least, very rarely

9

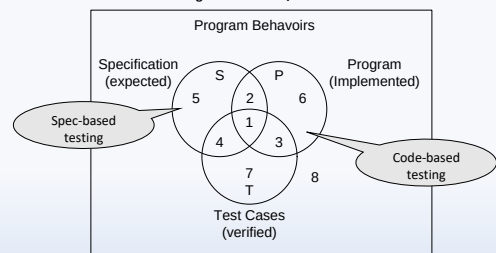
Testing and Program Behaviors



10

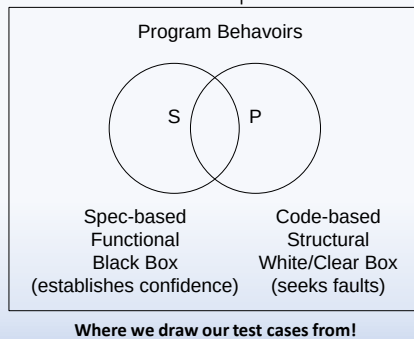
Testing and Program Behaviors

Which region is the best / most desired?



11

Tests Relation to Spec and Code



12

In Class Assignment

Step 1: Work to develop the test cases on *your* sheet *individually*

Step 2: Discuss your test cases amongst those around you. Identify a common set of test cases *based on what you were given only*.

Step 3: We will discuss the test cases together!

13

Test Cases

Code-Based	Spec-Based

14

Spec-Based

Create test cases (i.e., expected inputs and outputs) to test the following function that returns days in the month for the current year:

```
public int daysInMonth(int month)
```

The expected inputs are in the range of 1 to 12 for January to December, respectively. Values outside this range should return 0, since non-existent months have no days. Expected inputs should return the number of days in the month for the current year.

15

Code-Based

```
public int daysInMonth(int month) {
    int days = -1;
    switch (month) {
        // 31 day months
        case -1: case 1: case 3: case 5: case 7: case 8: case 10: case 12:
            days = 31;
            break;
        // 30 day months
        case 4: case 6: case 9: case 11:
            days = 30;
            break;
        // February
        case 2:
            days = 28;
            break;
    }
    return days;
}
```

16

"Full" Set of Test Cases

```
TestableCode test = new TestableCode(2019);
```

```
assertEquals(0, test.daysInMonth(-1));
assertEquals(0, test.daysInMonth(0));
assertEquals(31, test.daysInMonth(1));
assertEquals(28, test.daysInMonth(2));
assertEquals(31, test.daysInMonth(3));
assertEquals(30, test.daysInMonth(4));
assertEquals(31, test.daysInMonth(5));
assertEquals(30, test.daysInMonth(6));
assertEquals(31, test.daysInMonth(7));
assertEquals(31, test.daysInMonth(8));
assertEquals(30, test.daysInMonth(9));
assertEquals(31, test.daysInMonth(10));
assertEquals(30, test.daysInMonth(11));
assertEquals(31, test.daysInMonth(12));
assertEquals(0, test.daysInMonth(13));
```

```
test = new TestableCode(2020);
assertEquals(29, test.daysInMonth(2));
```

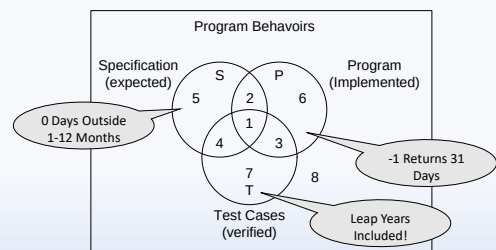
17

Types of Test Cases

- Included in **code**, not specification:
 - Month '-1' is returning 31 days.
- Included in **specification**, not code:
 - Non-existent months should return 0.
- Not *explicitly* included in either:
 - Leap years should be included if 'current year' is changed.

18

Testing and Program Behaviors



19

Psychology of Testing

- Proper (and correct) definition of testing depends upon the psychological attitude of the person doing the testing.
- Destructive nature of testing:
 - Testing is intended to cause a program to behave in a manner it was not intended or thought of by its designer or programmer.
 - Psychologically, programmers do not want to destroy their creations. They tend to select tests that will not demonstrate the presence of errors.
 - A programmer should avoid attempting to test his or her own program.

20

Objective of Testing

- In most cases, testing is conducted in an ad hoc fashion which occurs late in a software project, when time is short, budgets are nearly exhausted, managers are nervous, and customers are anxious.
- Testing strategies and techniques enable us to improve the thoroughness with which software tests are conducted and increase the probability that testing will achieve its primary objective:

**“to find the largest number of errors in the least amount of time
with a minimum allocation of resources”**

21